

Patrones de comportamiento

05

Cómo colaboran los objetos cuando el sistema deja de ser simple.

JUEVES · 26/02/2026 · 17:20

Observer
Command
Chain of Responsibility
Visitor
Mediator

La meta no es memorizar nombres

La meta es reconocer el problema que hay detrás.

- Explicar cómo se comunican los objetos en un sistema dinámico.
- Identificar qué responsabilidad conviene separar.
- Elegir un patrón y justificar también su costo.
- Leer y ejecutar un ejemplo TypeScript pequeño.

CRITERIO DE LOGRO

Puedes mirar un escenario y decir: “el problema es ____; usaría ____ porque ____”.

Cuatro momentos para pasar de la idea al código

- | | | |
|----|-----------------------------------|--|
| 01 | Dudas del proyecto | Aclaremos criterios y decisiones pendientes. |
| 02 | Patrones de comportamiento | Cinco formas de coordinar objetos. |
| 03 | Foro de la semana | Conectamos el tema con un problema real. |
| 04 | Dudas y comentarios | Cerramos con preguntas y aplicaciones. |

Dudas del proyecto

01

Antes de aprender un patrón, revisemos el problema que deben resolver.

idea → problema → solución → flujo → TypeScript

Los patrones de comportamiento organizan conversaciones

No describen solamente qué objetos existen; describen cómo colaboran.

Creacionales

cómo nacen los objetos

Estructurales

cómo se conectan

Comportamiento

cómo se coordinan

PREGUNTA GUÍA

¿Quién sabe qué? ¿Quién decide? ¿Quién recibe la solicitud?

El síntoma del código suele sugerir el patrón

Muchos objetos necesitan enterarse

Observer

suscripciones

Una acción puede venir de varios controles

Command

solicitud como objeto

Varias validaciones deben ejecutarse en orden

Chain of Responsibility

cadena de manejadores

Nuevos algoritmos sobre clases existentes

Visitor

separar datos y operaciones

Los controles se conocen demasiado

Mediator

un coordinador central

Un patrón se entiende antes de implementarlo

1. ¿Qué problema se repite?
2. ¿Qué parte cambia y quién debería conocerla?
3. ¿Qué roles colaboran y en qué orden?
4. ¿Qué costo acepto al aplicar la solución?

Explicar cómo se comunican los objetos en un sistema dinámico.

Identificar qué responsabilidad conviene separar.

Elegir un patrón y justificar también su costo.

Leer y ejecutar un ejemplo TypeScript pequeño.

SECUENCIA

Problema, roles, flujo, consecuencias. El nombre del patrón llega después.

Puedes mirar un escenario y decir: “el problema es ____; usaría ____ porque ____”.

Observer

01

Suscribir objetos a un flujo de eventos, sin acoplar el emisor a cada receptor.

idea → problema → solución → flujo → TypeScript

El problema no es notificar; es no conocer a todos los receptores

Si cada curso llamara directamente a cada estudiante, el curso dependería de nombres, métodos y cambios que no controla. La alternativa ingenua es preguntar constantemente si hay novedades: desperdicia tiempo y recursos.

SEÑAL

Polling repetido o una lista de clases concretas dentro del sujeto.

Observer: push y pull resuelven necesidades distintas

Push: el sujeto envía el dato en la notificación.

Pull: el observador recibe el evento y consulta el estado.

Push simplifica al observador; pull reduce el acoplamiento al formato del evento.

La elección depende de cuánto contexto necesitan los receptores.

PREGUNTA

¿El estudiante necesita el material completo, o solo saber que el curso cambió?

El sujeto publica; los observadores deciden qué hacer

- El sujeto mantiene una colección de observadores.
- Los observadores comparten un método como actualizar().
- Suscribir y cancelar ocurren en tiempo de ejecución.
- El sujeto envía el evento sin conocer la clase concreta.

IDEA MÍNIMA

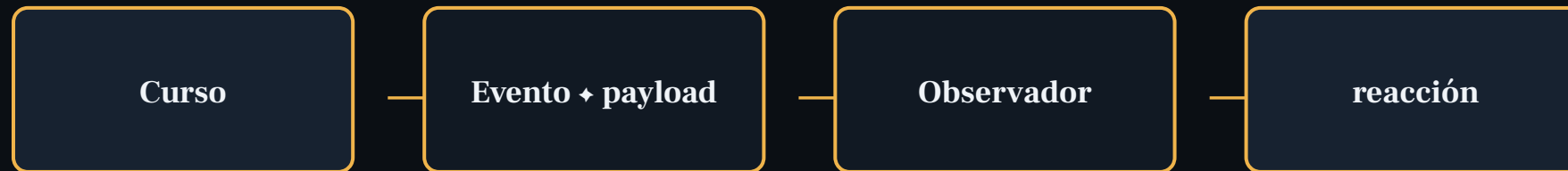
Una interfaz estable convierte muchos receptores concretos en una sola conversación.

El evento puede llevar contexto, no solo un aviso

Diseña la notificación para que el receptor tenga lo necesario.

EJEMPLO DE PAYLOAD

cursoId + título + fecha. Evita que el observador tenga que volver a consultar cinco servicios.



Un evento viaja a todos los suscriptores

Lee el diagrama como una conversación entre objetos.

FLUJO

publicarMaterial() → recorrer suscriptores → actualizar(mensaje)



El curso no necesita saber quién recibe el aviso

El segundo anuncio demuestra que cancelar una suscripción también es parte del patrón.

observer.ts

```
const curso = new Curso("ADS2");  
const ana = new Estudiante("Ana");  
const luis = new Estudiante("Luis");
```

```
curso.suscribir(ana);  
curso.suscribir(luis);  
curso.publicarMaterial("Ejercicio");
```

```
curso.cancelar(luis);  
curso.publicarMaterial("Solución");
```

CUÁNDO USARLO

Cuando muchos objetos reaccionan a un cambio y quieres agregar o quitar receptores sin tocar el emisor.

COSTO

Notificaciones inesperadas, orden de ejecución difícil de rastrear o demasiados observadores.

Pausa de 5 minutos: diseña una suscripción

Antes de ver el código, predice qué debería ocurrir.

1. Elige un emisor: curso, biblioteca o calendario.
2. Define dos observadores distintos.
3. Decide qué dato viaja en update().
4. ¿Qué pasa si uno cancela su suscripción?

COMPARTE

Explica qué clase conoce a quién y por qué el emisor no debe conocer la clase concreta.

Command

02

Convertir una solicitud en un objeto para poder reutilizarla, cambiarla o retrasarla.

idea → problema → solución → flujo → TypeScript

El mismo botón no debería conocer toda la lógica del portal

Un clic puede venir de un botón, un menú contextual, un atajo de teclado o una prueba automática. Si cada control implementa la operación, la lógica se duplica y la interfaz queda acoplada al negocio.

SEÑAL

Muchos controles repiten el mismo código o hay subclases como `BotonInscribir` y `BotonCancelar`.

Command no es solo una función: encapsula contexto

El comando puede guardar receptor, parámetros y metadatos.
Por eso se puede registrar, reintentar, poner en cola o deshacer.
El botón sigue siendo simple: solo sabe ejecutar().
El costo es introducir un objeto por cada acción importante.

PIENSA

¿Qué datos necesitaría guardar una acción de inscripción para poder deshacerla?

El emisor dispara; el comando conecta con el receptor

- El receptor contiene la operación real.
- El comando guarda receptor y parámetros.
- El emisor solo conoce ejecutar().
- Un mismo comando puede vincularse a varios emisores.

SEPARACIÓN

La interfaz deja de saber cómo se realiza la acción y solo sabe cuándo activarla.

Una solicitud se vuelve intercambiable

Lee el diagrama como una conversación entre objetos.

FLUJO

hacerClick() → ejecutar() → inscribir() / cancelar() → actualizar estado



El botón cambia de comportamiento sin cambiar su clase

La misma instancia de Boton puede ejecutar dos solicitudes distintas.

```
command.ts

const boton = new Boton(inscribir);

boton.hacerClic();
// Ana quedó inscrita en ADS2.

boton.cambiarComando(cancelar);
boton.hacerClic();
// Ana canceló su inscripción.
```

CUÁNDO USARLO

Cuando necesitas parametrizar acciones, ponerlas en cola, registrarlas o reutilizarlas desde varios controles.

COSTO

Más clases pequeñas y una operación indirecta: para una acción trivial puede ser demasiado.

Una misma acción puede vivir en varios controles

La barra, el atajo y una tarea automática comparten el mismo comando.

`command.ts`

```
const cmd = new InscribirCommand(portal, "Ana",  
  "ADS2");  
const boton = new Boton(cmd);  
const atajo = new Boton(cmd);  
  
boton.hacerClic();  
atajo.hacerClic();  
// La lógica de inscripción no se duplica.
```

EXTENSIÓN

Si el comando guarda el estado anterior, puede implementar deshacer().

PREGUNTA

¿Qué diferencia hay entre reutilizar el comando y duplicar el código en cada botón?

Chain of Responsibility

03

Pasar una solicitud por manejadores que pueden procesarla o detenerla.

idea → problema → solución → flujo → TypeScript

Las validaciones crecen hasta convertirse en un método imposible de mantener

Autenticación, prerequisites, cupos, pagos y permisos suelen agregarse uno detrás de otro. Cuando todo vive en un único método, modificar una regla puede afectar las demás y reutilizar solo una parte se vuelve difícil.

SEÑAL

if / else anidados, orden rígido y comprobaciones duplicadas.

Cada regla conoce solo al siguiente eslabón

- Cada manejador tiene una responsabilidad pequeña.
- La cadena se arma durante la configuración.
- Un manejador puede detener la solicitud.
- Agregar una regla no exige reescribir las anteriores.

DECISIÓN

¿La solicitud continúa o termina aquí? Esa decisión pertenece al manejador actual.

La cadena puede validar todo o encontrar un único responsable

Cadena de validación: cada manejador ejecuta su regla y, si pasa, delega al siguiente.

Manejador exclusivo: el primero que puede resolver la solicitud la detiene.

Ambas variantes comparten la misma idea: desacoplar quién recibe la solicitud.

La diferencia está en cuándo termina el recorrido.

DECISIÓN

¿Necesitas acumular comprobaciones o encontrar un solo manejador?

La solicitud avanza hasta que una regla la detiene

Lee el diagrama como una conversación entre objetos.

FLUJO

`manejar(solicitud) → validar() → siguiente.manejar() o false`



La primera validación fallida evita trabajo innecesario

La solicitud de Ana se detiene; la de Luis recorre toda la cadena.

chain-of-responsibility.ts

```
const cadena = crearCadenaDeInscripcion();

cadena.manejar({
  estudiante: "Ana", autenticado: false,
  tienePrerequisito: true, cuposDisponibles: 3,
});
// Rechazada: no está autenticada.

cadena.manejar({ ...solicitudDeLuis });
// Aprobada: Luis puede inscribirse.
```

CUÁNDO USARLO

Cuando varias reglas pueden manejar una solicitud y quieres componer el orden sin una clase central gigante.

COSTO

El flujo puede volverse difícil de seguir y una solicitud podría no ser manejada por ningún eslabón.

El orden de la cadena también es una decisión de diseño

Una regla barata y decisiva suele ir antes que una comprobación costosa.

RAZONAMIENTO

Si autenticación falla, no tiene sentido consultar cupos. Si una validación llama a una API lenta, piensa dónde conviene colocarla.



Visitor

04

Separar un algoritmo de los objetos sobre los que trabaja.

idea → problema → solución → flujo → TypeScript

Los datos académicos no deberían crecer cada vez que aparece un nuevo reporte

Curso y Estudiante representan información del dominio. Exportar a CSV, generar un resumen o validar datos son comportamientos distintos que pueden cambiar con frecuencia.

SEÑAL

Clases de datos llenas de métodos para exportar, imprimir, contar y validar.

El objeto acepta al visitante y selecciona la operación correcta

- El elemento académico expone aceptar(visitor).
- El visitante ofrece visitarCurso() y visitarEstudiante().
- Cada clase elige el método adecuado: doble despacho.
- Un nuevo reporte se agrega como otro visitante.

IDEA DIFÍCIL

No es solo pasar un objeto: el elemento participa en la selección del método concreto.

Visitor decide el método correcto en dos pasos

El elemento conoce su tipo; el visitante conoce la operación.

PASO A PASO

`Curso.aceptar(csv)` llama a `csv.visitarCurso(this)`. Así no necesitas un switch por tipo.



El mismo conjunto de objetos puede recibir distintos algoritmos

Lee el diagrama como una conversación entre objetos.

FLUJO

`elemento.aceptar(visitor) → visitor.visitarTipo(elemento)`



Agregar un reporte no obliga a modificar Curso ni Estudiante

El segundo visitante cuenta elementos usando las mismas clases académicas.

visitor.ts

```
const elementos = [curso, estudiante];
```

```
const csv = new ExportarCsvVisitor();
for (const elemento of elementos) {
  elemento.aceptar(csv);
}
console.log(csv.resultado());
```

```
const conteo = new ContarElementosVisitor();
for (const elemento of elementos) {
  elemento.aceptar(conteo);
}
```

CUÁNDO USARLO

Cuando la estructura de objetos es estable y los algoritmos sobre ella cambian o se multiplican.

COSTO

Añadir un nuevo tipo de elemento obliga a actualizar todos los visitantes existentes.

Visitor es potente cuando la estructura cambia poco

Agregar un reporte nuevo es barato: creas otro visitante.

Agregar un tipo nuevo de elemento es caro: debes actualizar todos los visitantes.

Por eso conviene usarlo cuando la jerarquía de elementos es estable.

Si cambian ambas cosas todo el tiempo, busca otra solución.

CONTRASTE

Visitor favorece agregar operaciones; no favorece agregar tipos de datos.

Mediator

05

Centralizar la coordinación para reducir dependencias directas entre objetos.

idea → problema → solución → flujo → TypeScript

Cuando todos los controles se hablan, cada cambio tiene demasiados efectos

En un formulario, el curso habilita horarios, el horario afecta el botón y el botón valida todos los campos. Si cada control conoce a los demás, el sistema se vuelve una red de dependencias.

SEÑAL

componentes con referencias cruzadas y lógica de coordinación repartida.

Mediator y Observer desacoplan, pero no hacen lo mismo

Observer distribuye un evento a muchos suscriptores.

Mediator coordina una conversación entre colegas y decide qué acción ocurre.

Observer pregunta “¿quién quiere enterarse?”.

Mediator pregunta “¿qué debe coordinarse después de este evento?”.

PISTA

Si hay un centro que conoce las reglas del diálogo, probablemente estás viendo un Mediator.

Los componentes notifican; el mediador decide qué ocurre

- Los controles dejan de invocarse directamente.
- Cada componente conoce solo al mediador.
- El mediador coordina estados y validaciones.
- La lógica del diálogo queda visible en un solo lugar.

REGLA

El mediador no reemplaza toda la lógica: organiza la colaboración entre colegas.

El formulario convierte muchos mensajes directos en una conversación central

Lee el diagrama como una conversación entre objetos.

FLUJO

`curso.seleccionar() → mediador.notificar() → habilitar horario → guardar`



El botón no valida cada campo: pregunta al formulario

Los controles colaboran a través de FormularioInscripcion.

mediator.ts

```
guardar.hacerClic();  
// No se puede guardar: faltan datos.  
  
curso.seleccionar("ADS2");  
// El formulario habilitó horarios.  
  
horario.seleccionar("Jueves 17:20");  
guardar.hacerClic();  
// Inscripción guardada.
```

CUÁNDO USARLO

Cuando un conjunto de objetos interactúa de forma compleja y conviene tener un coordinador explícito.

COSTO

El mediador puede convertirse en una clase demasiado grande si absorbe responsabilidades ajenas.

Pausa de 7 minutos: reconstruye el formulario

Dibuja el flujo antes de mirar la implementación.

1. Nombra tres controles del portal.
2. Escribe un evento que cada control pueda notificar.
3. Decide qué reglas viven en el mediador.
4. Marca una validación que debe detener el guardado.

COMPARTE

Compara tu diseño con otra pareja: ¿qué dependencias directas eliminaron?

Cada patrón mueve una responsabilidad a un lugar distinto

Observer

Quién recibe un cambio

suscripciones

Command

Qué solicitud ejecutar

objeto comando

Chain

Quién procesa o detiene

manejadores encadenados

Visitor

Qué algoritmo aplicar

visitante externo

Mediator

Quién coordina colegas

objeto mediador

Un portal puede combinar patrones

Observer

Curso publica material

estudiantes reciben

Command

Botón solicita inscripción

comando ejecuta

Chain

Inscripción pasa por reglas

validaciones encadenadas

Visitor

Cursos reciben reportes

operación externa

Mediator

Formulario coordina campos

diálogo central

No elijas el patrón por el nombre: explica el problema

Usa una frase mínima para defender tu decisión.

FRASE DE DECISIÓN

“Elegiría ____ porque el problema es ____; acepto el costo de ____.”

Caso

El portal debe avisar a estudiantes, permitir que un botón ejecute una acción, validar una inscripción y generar un reporte.

¿Qué patrón resuelve cada parte? ¿Cuál sería innecesario?

Diseñemos el flujo de una inscripción

Trabajen en parejas y hagan visible la conversación entre objetos.

Identifica la parte del sistema que concentra demasiada responsabilidad.

Elige un patrón de comportamiento y justifica por qué.

Dibuja las interfaces y el flujo principal.

Escribe el método clave en TypeScript.

ENTREGA RÁPIDA

Un diagrama con 3–5 objetos + una explicación de 60 segundos.

Foro de la semana

03

Piensa en una clase que “lo hace todo” o en un método lleno de if.

Lleva el catálogo a un proyecto que ya conoces.

¿Qué patrón de comportamiento podría ayudar? ¿Qué costo aceptarías al introducirlo?

idea → problema → solución → flujo → TypeScript

Publica el escenario, el patrón elegido y una razón concreta.

Los patrones son formas de conversar

Observer · Command · Chain · Visitor · Mediator

¿Qué problema de colaboración aparece en tu proyecto?

¡Gracias por su atención!